SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Duration : 1 Hour

Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I **_M.Greeshma/192524195** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Greeshma

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Water Jug Problem

Problem: Given a 4-gallon jug and a 3-gallon jug, obtain exactly 2 gallons of water in the 4-gallon jug. There are no measurement markings on either jug.

Solution:

Let the state be represented as \((x,y)\), where \(x\) is the amount of water in the 4-gallon jug and \(y\) is the amount in the 3-gallon jug.

Initial state: (0, 0) — both jugs are empty.

Step	**Action**	**State (4-gallon, 3-gallon)**
1	Fill the 3-gallon jug	(0, 3)
2	Pour the 3-gallon jug into the 4-gallon jug	(3, 0)
3	Fill the 3-gallon jug again	(3, 3)
4	Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full	(4, 2)
5	Empty the 4-gallon jug onto the ground	(0, 2)
6	Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug	(2, 0)

Result: Exactly 2 gallons of water are present in the 4-gallon jug.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the

following questions.

i. What are the percept's for this agent?

ii. Characterize the operating environment.

iii. What are the actions the agent can take?

iv. How can one evaluate the performance of the agent?

v. What sort of agent architecture do you think is most suitable for this agent?

Why?

Mars Rover as an Intelligent Agent

A Mars Rover is an autonomous robotic agent that explores the surface of Mars, collects scientific samples, and sends information to Earth.

i. What are the percepts for this agent?

Percepts are the information the agent receives from its sensors.

- Images and video from cameras.
- Distance and obstacle information from sensors.
- Soil and rock composition from scientific instruments.
- Temperature, pressure, and environmental conditions.
- Rover position, orientation, speed, and battery level.
- Signals indicating equipment status and communication availability.

ii. Characterize the operating environment

The Mars Rover operates in an environment with the following characteristics:

- Partially observable: It cannot directly observe every feature of the Martian surface.

- Stochastic: Outcomes can be uncertain because of terrain, equipment, and environmental conditions.
- Sequential: Each movement or sampling decision affects future decisions.
- Dynamic: Environmental conditions and operational risks can change.
- Continuous: Position, time, distance, and many sensor measurements are continuous.
- Single-agent: The rover performs its mission independently, although Earth-based operators can provide instructions.

iii. What actions can the agent take?

- Move forward, backward, and turn.
- Capture images and scan the surroundings.
- Collect and analyse soil and rock samples.
- Avoid obstacles and select safe routes.
- Recharge or conserve battery power where possible.
- Communicate scientific findings to Earth.
- Adjust its plan according to sensor readings and mission priorities.

iv. How can the performance of the agent be evaluated?

Performance can be evaluated using:

- Accuracy of sample analysis.
- Amount of useful scientific data collected.
- Successful completion of mission objectives.
- Safety of navigation and obstacle avoidance.
- Efficient use of energy and time.
- Reliability of communication with Earth.
- Minimization of equipment damage and mission failures.

v. What is the most suitable agent architecture? Why?

Model-based, goal-based, and utility-based intelligent agent architecture is most suitable.

- A model-based agent maintains an internal representation of its surroundings, including obstacles and terrain.
- A goal-based agent selects actions that help achieve objectives, such as reaching a sample location.
- A utility-based agent compares possible actions based on safety, scientific value, time, and energy consumption.

Combining these capabilities allows the rover to navigate uncertain terrain, make informed decisions, and prioritize scientific goals while maintaining safety.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Eight Queens Problem

Problem: Place eight queens on an \(8 \times 8\) chessboard such that no two queens attack each other.

A queen can attack another queen in the same row, column, or diagonal.

A. Formulation of the problem components

1. Initial state: An empty chessboard.
2. State space: All possible configurations of queens placed on the board.

3. Actions: Place a queen in a safe square, usually one queen per column.
4. Transition model: The board changes when a queen is placed in a valid square.
5. Goal test: All eight queens are placed without any two queens attacking each other.
6. Path cost: Each queen placement has a cost of 1, making the total cost 8 for a complete solution.

B. Search strategies

- Breadth-First Search (BFS): Explores configurations level by level, but may consume substantial memory.
- Depth-First Search (DFS): Explores one configuration deeply before backtracking.
- Backtracking search: Places a queen in a safe position, proceeds to the next column, and returns to an earlier column whenever no safe position exists.

Backtracking is particularly suitable because it rejects invalid configurations early.

C. Algorithm / pseudocode

```
N_QUEENS(board, column):
  if column == 8:
    return TRUE

  for row = 0 to 7:
    if IS_SAFE(board, row, column):
      board[row][column] = 1

      if N_QUEENS(board, column + 1):
```

```
            return TRUE

        board[row][column] = 0

    return FALSE
```

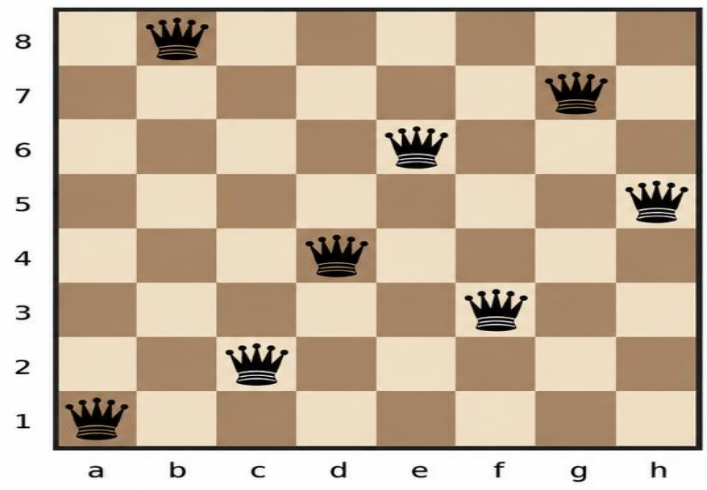
```
IS_SAFE(board, row, column):
    Check the same row on the left
    Check the upper-left diagonal
    Check the lower-left diagonal

    If no queen is found:
        return TRUE
    Otherwise:
        return FALSE
```

D. One valid solution

The following arrangement gives the row number of the queen in each column:

[[1,5,8,6,3,7,2,4]]



4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

OLA Cab Booking Using a Problem-Solving Agent

Problem: A customer wants to travel from a source location to a destination using the OLA Cab booking application. The customer can select a cab type such as Mini, Micro, Sedan, Shared, or Prime.

A. Type of agent

A goal-based problem-solving agent is suitable because it identifies the customer's destination as the goal and searches for an appropriate cab and route.

A utility-based approach can also be incorporated to compare fares, comfort, travel time, and customer preferences.

B. Problem components

1. Initial state: Customer's current location.
2. Goal state: Reaching the destination.
3. Actions: Select a cab, choose a route, and book the cab.
4. Constraints: Cab availability, fare, travel time, and customer preferences.
5. Performance measure: Successful booking, safe arrival, reasonable fare, and minimum travel time according to the customer's preferences.

C. Pseudocode

```
OLA_CAB_BOOKING():

    INPUT source_location
    INPUT destination
```

```
INPUT preferred_cab_type
INPUT budget

IF source_location == destination:
   PRINT "Source and destination are the same"
   RETURN

available_cabs = SEARCH_AVAILABLE_CABS(
   source_location,
   destination
)

IF available_cabs is empty:
   PRINT "No cabs available"
   RETURN

suitable_cabs = FILTER available_cabs
   BY preferred_cab_type
   AND budget

IF suitable_cabs is empty:
   PRINT "No suitable cab available"
   RETURN

selected_cab = SELECT_BEST_CAB(suitable_cabs)
```

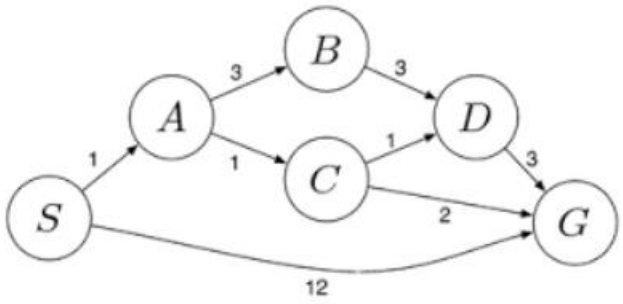
```
estimated_fare = CALCULATE_FARE(
    selected_cab,
    source_location,
    destination
)

DISPLAY selected_cab
DISPLAY estimated_fare

IF customer_confirms_booking:
    CONFIRM_BOOKING()
    PRINT "Cab booked successfully"
ELSE:
    PRINT "Booking cancelled"
```

Result: The agent searches for available cabs, selects a suitable option according to the customer's preferences and budget, and completes the booking after confirmation.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm. The delivery network is represented as a weighted graph:



Uniform Cost Search (UCS) for the Delivery Network

Problem: Find the minimum-cost path from the starting warehouse \(S\) to the destination warehouse \(G\).

Uniform Cost Search expands the node with the lowest cumulative path cost first. It uses a priority queue ordered by path cost.

A. Given weighted graph

The edge costs are:

Route	**Cost**
S → A	1
S → G	12
A → B	3
A → C	1
B → D	3
C → D	1
C → G	2
D → G	3

B. UCS exploration

Step	Node expanded	Path cost \(g(n)\)	Frontier after expansion (lowest cost first)
1	S	0	A(1), G(12)
2	A	1	C(2), B(4), G(12)
3	C	2	D(3), B(4), G(4)
4	D	3	B(4), G(4), G(6)
5	G	4	Goal reached

When \(G\) is removed from the priority queue at cost 4, UCS terminates. If nodes with equal costs are handled in a different order, \(B\) may be expanded before \(G\); the optimal cost remains unchanged.

C. Cost calculation

The optimal route is:

[S \rightarrow A \rightarrow C \rightarrow G \]

Total cost:

[\begin{aligned} \text{Cost} &= 1+1+2\\ &=\boxed{4} \end{aligned} \]

Other possible routes include:

- (S \rightarrow A \rightarrow C \rightarrow D \rightarrow G\): cost (1+1+1+3=6\).
- (S \rightarrow A \rightarrow B \rightarrow D \rightarrow G\): cost (1+3+3+3=10\).
- (S \rightarrow G\): cost \(12\).

D. UCS pseudocode

```
UNIFORM_COST_SEARCH(start, goal):


    priority_queue = [(0, start, [start])]
```

```
  visited_cost = {}

  WHILE priority_queue is not empty:

    (cost, node, path) = REMOVE_LOWEST_COST()

    IF node == goal:
      RETURN path, cost

    IF node in visited_cost
      AND visited_cost[node] <= cost:
      CONTINUE

    visited_cost[node] = cost

    FOR each neighbor of node:
      new_cost = cost + EDGE_COST(node, neighbor)

      IF neighbor not in visited_cost
        OR new_cost < visited_cost[neighbor]:
        INSERT(
          new_cost,
          neighbor,
          path + [neighbor]   )
RETURN "No path found"
```

Result: Uniform Cost Search finds the least-cost path \(S \rightarrow A \rightarrow C \rightarrow G\), with a minimum total cost of 4 units.